



ÇANKAYA UNIVERSITY
Software Engineering Department



SENG 491 – 492 Graduation Project
Software Design
Description

Dynamic Port Switching

Eren Abdullah BEŞİK – 202128019

Şahin ERŞAN – 202128002

Berfin Duru ALKAN – 202228005

Version 1.0

Table of Contents

1	Introduction.....	1
1.1	Purpose.....	1
1.2	Scope	1
1.3	Definitions, acronyms, and abbreviations	1
1.4	Overview	2
1.5	Version History	2
1.6	Operational Assumptions and Out-of-Band Configuration.....	2
2	Architecture.....	3
2.1	Hardware Requirements	3
2.2	Software & Environment Requirements	4
3	User Interface	4
3.1	Mode: Peer Startup / Configuration.....	4
3.1.1	Manual Mode (TCP).....	5
3.1.2	Manual Mode (UDP).....	6
3.1.3	Automatic Mode	7
4	High Level Design.....	8
4.1	Module 1 — Communication Controller.....	8
4.2	Module 2 — Protocol Decision Logic.....	8
4.3	Module 3 — Port Management Component.....	9
4.4	Module 4 — Timing & Synchronization Handler	9
4.5	Module 5 — Role Assignment Component.....	10
4.6	Module 6 — Logging Component.....	10
4.7	Module 7 — Message/Package Format Specification	10
5	Low Level Design	11
5.1	Communication Controller (Flow)	11
5.2	Protocol Decision Logic (Flow)	11
5.3	Port Management Component (Flow)	11
5.4	Timing & Synchronization Handler (Flow)	12
5.5	Logging Component (Flow).....	12
5.6	Role Assignment Component (Flow).....	12
5.7	Packet Reliability Mechanisms (UDP Mode)	12
6	References	13

1 Introduction

1.1 Purpose

The purpose of this Software Design Description (SDD) is to define the technical design of the **P2P Dynamic Port Switching** system, including its architecture, modules, interfaces, and implementation-level structure. This document is intended for developers and reviewers who need to understand how the system will be built in **C (low-level / “raw” socket usage)** and how protocol components interact (port computation, role assignment, connection handling, authentication, optional encryption, and logging).

1.2 Scope

This SDD covers the design of a **two-peer** (Peer A & Peer B) communication system where both peers dynamically switch ports based on a shared secret and time-based deterministic computation. The design includes:

- Peer-to-peer architecture (no central application server required)
- Transport support for **TCP and UDP**
- Module decomposition and responsibilities (high level + low level flows)
- Logging, configuration, and error handling

This SDD does **not** cover:

- GUI screens (this is a nice to have feature)
- Large-scale multi-peer mesh networking

1.3 Definitions, acronyms, and abbreviations

TERM	DEFINITION
P2P	Peer-to-peer communication with no central controller
MTD	Moving Target Defense; changing attack surface over time
Synchronization Secret Key	Shared secret used to compute ports/tokens/keys
Listener/Dialer	Deterministic roles each interval to avoid collisions
NIC (Network Interface Card)	Hardware network adapter enabling TCP/IP networking
Raw C / Low-level sockets	Implementing protocol using C sockets with minimal abstraction (POSIX/WinSock), enabling fine-grained control

1.4 Overview

- Section 2 describes Architectural Requirements.
- Section 3 provides the User Interface (CLI) command modes and outputs.
- Section 4 provides High-Level Design and module decomposition.
- Section 5 provides Low-Level Design and detailed algorithmic flows.

1.5 Version History

Provide a table of table of changes to this document

Version No	Description of change	Date
1.0	Initial Release	03.12.2025

1.6 Operational Assumptions and Out-of-Band Configuration

To ensure the deterministic nature of the Dynamic Port Switching algorithm and to mitigate common cryptographic bootstrapping issues, the system relies on a **Pre-Shared Configuration (PSC)** model. The following parameters must be exchanged between Peer A and Peer B via a secure, secondary channel (Out-of-Band) before the application is initialized:

- **IP Address Knowledge:** Both peers must know each other's network addresses (IPv4) beforehand. The current version of the system does not utilize a discovery server or STUN/TURN services for initial peer location.
- **Secret Key Exchange:** The Randomization-Secret-Key used for SHA256 port and role calculations is assumed to be pre-shared. This prevents an "Eavesdropper" from calculating the next port by simply observing the initialization phase, as the key is never transmitted over the active network.
- **Unique Peer ID Assignment:** To avoid "Role Collision" (where both peers attempt to take the same network role simultaneously), users must manually assign unique IDs (0 and 1) during setup. One peer must be configured as ID 0 and the other as ID 1 to ensure that the deterministic role assignment logic (Module 5) functions correctly without negotiation.
- **Parameter Synchronization:** Peers must agree on the Timing-Interval and the Port-Range. A mismatch in these parameters will lead to synchronization failure, as the calculated target ports will not align.

2 Architecture

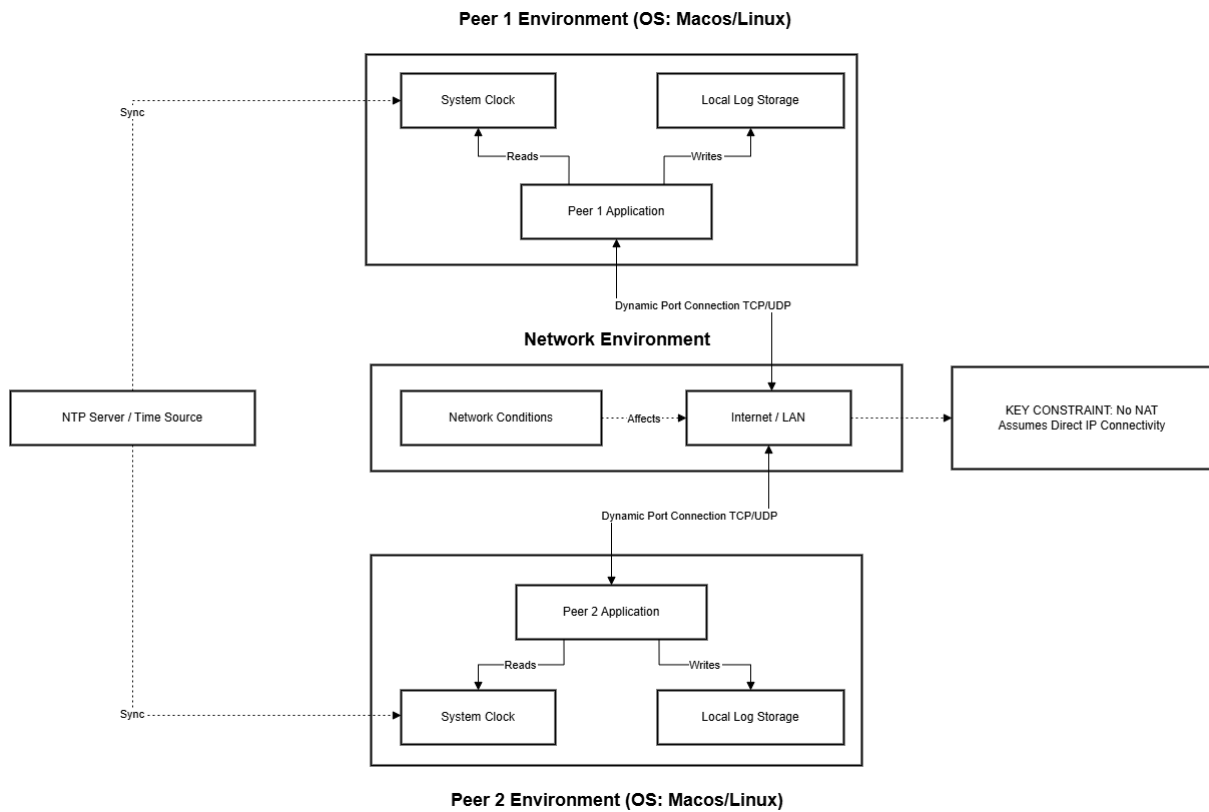


Diagram 1: Physical Architecture Diagram

2.1 Hardware Requirements

Component	Minimum Requirement	Rationale
Processor (CPU)	Any standard x86 or ARM processor (1 GHz or faster)	Handling computations and real-time log updates
Memory (RAM)	1 GB System RAM (Application footprint: < 50 MB)	While the application itself requires minimal memory (<100 MB), the host OS needs sufficient RAM to maintain stable network operations.
Storage (Disk)	50 MB Free Space	Required for application binaries and storing local execution logs.
Network Interface	Ethernet or Wi-Fi Adapter (NIC)	Must support standard TCP/IP protocol suite and full-duplex communication.

2.2 Software & Environment Requirements

- **Operating System (OS):**
 - **Primary Support:** Unix-based systems (Linux distributions such as Ubuntu, Debian, Fedora, Kali Linux) and macOS.
 - **Note on Windows:** Due to OS-level restrictions on Raw Socket creation (specifically regarding TCP headers), native Windows support is excluded. Windows users are advised to run the application within a Virtual Machine (VM) or WSL2 (Windows Subsystem for Linux).
- **Network Protocol Support:** The operating system must natively support the **TCP/IP Protocol Suite** (IPv4).
- **Time Synchronization:** Active **NTP (Network Time Protocol)** service is mandatory. The system clock must be synchronized to ensure the drift remains within the tolerated *timing-interval*.
- **User Privileges:** The application requires **Administrator (Windows)** or **Root/Sudo (Linux/macOS)** privileges to access Raw Sockets and bind to specific ports dynamically.
- **Dependencies:**
 - C Runtime Library (CRT) for the target OS.
 - System libraries for GUI rendering (e.g., GTK, Qt, or native Windows API depending on implementation).

3 User Interface

The system is CLI-based. Therefore, “screens” are represented as CLI command modes and outputs.

3.1 Mode: Peer Startup / Configuration

Title: Peer Launch Interface (CLI)

3.1.1 Manual Mode (TCP)

INPUTS

- **Initial Setup Inputs:**
 - Target IP Address: Target Device's IP address. (should be known)
 - Timing-Interval: The duration (in seconds) between each dynamic port switch event. Both peers must provide the identical value to ensure their time-step calculations remain synchronized.
 - Port Range: The numerical bounds (Minimum and Maximum port numbers) within which the dynamic ports will be generated. Both peers must utilize the same range to ensure the deterministic mapping produces matching target ports.
 - Randomization-Secret-Key: This key is needed for randomize port generation.
 - Peer ID: A unique identifier (0 or 1) assigned to the peer.
 - This ID is used by the system to deterministically calculate the peer's role (Listener or Dialer) for each specific timing-interval.
 - While the ID remains constant, the network role may rotate to increase unpredictability against attackers.
 - Protocol: TCP.
- **Runtime Inputs:**
 - Chat Message: After Initial Setup user starts chatting with the other peer

OUTPUTS

- **System Status Indicator:**
 - Status: This shows the current status of the program. (Connected/Disconnected)
 - Current Port: This shows the current running port.
 - Next Switch in: This shows how much time left to next switch event.
- **Background Activity Logs (Console/Debug):**
 - **Description:** A real-time log stream displayed within the application console. This output provides visibility into the system's background logic without obstructing the primary chat interface.
 - **Output Content:**
 - **State Transitions:** Notifications when the system transitions between *Listener* and *Dialer* roles.
 - **Port Hopping Events:** Alerts triggering upon Timing-Interval timeout, displaying the newly calculated **Target Port** (e.g., [INFO] Interval Reached. Switching to Port: 25444).
 - **Connection Status:** Updates regarding socket teardown and re-establishment.
 - **Error Alerts:** Critical network failures or timeout warnings (excluding sensitive authentication data).

3.1.2 Manual Mode (UDP)

INPUTS:

- **Initial Setup:**
 - Target IP Address: Target Device's IP address.(should be known)
 - Timing-Interval: The fixed time window (in seconds) used to trigger a port hopping event. It serves as a constant in the deterministic algorithm for calculating the Time-Step.
 - Port Range: Defines the specific pool of available ports (e.g., 20000 to 30000) used for stateless packet transmission.
 - Randomization-Secret-Key: This key is needed for randomize port generation
 - Peer ID: A unique identifier (0 or 1) assigned to the peer.
 - This ID is used by the system to deterministically calculate the peer's role (Listener or Dialer) for each specific timing-interval.
 - While the ID remains constant, the network role may rotate to increase unpredictability against attackers.
 - Protocol: UDP.
- **Runtime Input:**
 - Chat Message: After Initial Setup user starts chatting with the other peer.

OUTPUTS:

- **Chat History Panel:**
 - **Sender:** The outgoing message appears on the interface immediately upon sending.
 - **Receiver:** The incoming message appears only if the packet is successfully received (subject to UDP packet loss).
- **Delivery Status (Key Distinction):**
 - Due to the connectionless nature of UDP, "Read" or "Delivered" receipts are not available. The status indicator will only display "**Sent**" once the packet leaves the interface.
- **Current Target Port:**
 - **Target Port: 24005** (Displays the specific dynamic port number currently calculated and targeted for transmission).
- **Background Activity Logs (Console/Debug) - UDP Mode**
 - Description: A real-time log stream displayed within the application console. This output provides visibility into the system's background logic, specifically tracking stateless packet dispatch and dynamic port targeting without obstructing the primary chat interface.
 - **Output Content:**
 - Port Hopping Events: Alerts triggering upon Timing-Interval timeout, displaying the newly calculated Target Port for the next batch of packets (e.g., [INFO] Interval Reached. New Target Port: 28999).
 - Transmission Status: Real-time confirmations of packet dispatch to the calculated dynamic ports. Unlike TCP, this indicates distinct "fire-and-forget" events rather than a session status (e.g., [UDP] Packet dispatched to 192.168.1.50:28999).

- Error Alerts: Socket transmission failures, binding errors, or "Network Unreachable" warnings (excluding delivery guarantees, as UDP does not provide acknowledgments).

3.1.3 Automatic Mode

INPUTS

- Initial Setup (Configuration):
 - Standard Parameters: Target IP Address, Randomization-Secret-Key, Peer ID.
 - Mode Selection: Auto (Selected).
 - Decision Thresholds (Advanced Config): Latency limit (e.g., >200ms) or Packet Loss rate (e.g., >5%) used to trigger protocol switching. *(Note: These are typically hidden in advanced settings or hardcoded defaults).*
- Runtime Input:
 - Chat Message: Continuous text input for the message stream.

OUTPUTS

- Chat History Panel:
 - A seamless flow of incoming and outgoing messages. The interface remains consistent even when the underlying transport protocol changes.
- System Notifications:
 - Real-time alerts regarding protocol adaptation logic.
 - *Example:* [INFO] Network unstable. Switching to UDP mode.
 - *Example:* [INFO] Network conditions improved. Reverting to TCP.

Background Activity Logs (Console/Debug)

- **Description:** A real-time log stream displayed within the application console or GUI debug panel. In Adaptive Mode, this is critical for monitoring the system's decision-making process as it toggles between protocols based on network stability.
- **Output Content:**
 - **Adaptation Events:** Notifications when the system decides to switch protocols based on the defined thresholds (e.g., [AUTO] High Latency (250ms) detected. Switching to UDP).
 - **Port Hopping Events:** Alerts triggering upon Timing-Interval timeout, displaying the newly calculated **Target Port** (e.g., [INFO] Interval Reached. New Target Port: 25444).
 - **Operational Status (Hybrid):** Depending on the active protocol, displays either TCP connection states or UDP packet dispatch confirmations:
 - *If TCP:* [TCP] Re-handshake successful on port 25444.
 - *If UDP:* [UDP] Packet dispatched to port 25444.
 - **Error Alerts:** Network failures or threshold breaches that trigger a protocol switch (e.g., [WARN] TCP Connection Timed Out. Fallback to UDP initiated).

4 High Level Design

At a high level, both peers run the same executable and follow the same deterministic algorithm to:

- compute port and role,
- establish a TCP/UDP connection,
- perform mutual authentication,
- exchange data,
- optionally rotate secret/session key,
- log and continue.

4.1 Module 1 — Communication Controller

Brief Description: This module acts as the central orchestrator for the Transport Layer operations. It is responsible for maintaining the state of the active protocol (TCP or UDP) and managing the transition between them. It executes the actual switching logic based on commands from the *Protocol Decision Logic* module or user manual input.

Responsibilities:

- **Protocol Lifecycle Management:** Handles the initialization, binding, and teardown of TCP sockets and UDP endpoints.
- **State Retention:** Stores the currently active protocol flag (e.g., ACTIVE_PROTO = TCP) to ensure the rest of the system utilizes the correct transport method for the current interval.
- **Switch Execution:**
 - *Manual Switch:* Instantly changes protocol upon user request via CLI/GUI.
 - *Automatic Switch:* Executes protocol transitions triggered by the adaptive logic, ensuring seamless handover (e.g., draining TCP buffers before switching to UDP).

4.2 Module 2 — Protocol Decision Logic

Brief Description: This module implements the intelligence behind the "Adaptive Mode." It continuously analyzes network conditions (latency, packet loss) and uses the **Randomization-Secret-Key** to make probabilistic decisions regarding which protocol to use for the next interval.

Responsibilities:

- **Network Analysis:** Monitors round-trip time (RTT) and packet delivery success rates to classify the network state (e.g., *Stable*, *Congested*, *Critical*).
- **Dynamic Weight Adjustment:** Adjusts the selection probability ratio based on network classification.
 - *Example:* Under optimal conditions, the selection ratio might be **50% TCP / 50% UDP**.
 - *Example:* Under high traffic/congestion, the system automatically skews the ratio to **20% TCP / 80% UDP** to favor connectionless transmission.

- **Deterministic Decision Making:** Uses the shared Randomization-Secret-Key combined with the current Time-Step and the calculated Weight to deterministically select the protocol. This ensures that both peers—observing the same network conditions—arrive at the same protocol decision without explicit negotiation.

4.3 Module 3 — Port Management Component

Brief Description: This module is responsible for the execution mechanism of the Dynamic Port Switching logic. Unlike the decision modules, this component handles the actual "heavy lifting" of binding to new ports and managing the transition lifecycle.

Responsibilities:

- **Deterministic Port Computation:** Calculates the active communication port for the current interval using the Randomization-Secret-Key and Time-Step. It supports both pseudo-random selection (based on cryptographic hashing) and range-based mapping.
- **Re-Rendezvous Execution:** Manages the critical "Handover Phase" where the system transitions from the old port to the new port. This involves tearing down the previous socket and initializing the new one.
- **Connection Verification:** Enforces a "Verify-then-Transmit" policy. Even if a port is open, the system halts application data transmission until the mutual handshake on the new port is fully verified.

4.4 Module 4 — Timing & Synchronization Handler

Brief Description:

This module acts as the "heartbeat" of the system. It abstracts the operating system's time functions and ensures that the application runs in lock-step with the configured time intervals.

Responsibilities:

- **NTP Reliance:** Relies on the host Operating System's Network Time Protocol (NTP) service. It assumes that both peers are reasonably synchronized to a global clock source.
- **Time-Step Calculation:** continuously calculates the current Time-Step index($T = \text{CurrentTime} / \text{Interval}$).
- **Event Triggering:** Serves as an interrupt generator. It monitors the clock and triggers signals for:
 - **Port Hopping:** When the Time-Step increments.
 - **Protocol Re-evaluation:** Periodically triggering the *Protocol Decision Logic* to check if network conditions have changed.

4.5 Module 5 — Role Assignment Component

Brief Description: This module deterministically assigns the network role (Listener or Dialer) to each peer for every time-step. It ensures that for any given interval, one peer acts as the server while the other acts as the client without manual negotiation.

Responsibilities:

- **Deterministic Role Calculation:** Uses the Randomization-Secret-Key and Time-Step to calculate which peer ID should listen and which should dial.
- **Collision Avoidance:** Guarantees that both peers do not attempt to take the same role simultaneously, preventing connection failures.

4.6 Module 6 — Logging Component

Brief Description: A specialized monitoring module designed to capture high-value operational metrics and state transitions without recording sensitive authentication secrets. It implements the Hybrid Logging mechanism (Console + File).

Responsibilities:

- **State Transition Logging:** Records every instance of a protocol switch (e.g., TCP -> UDP) and the rationale behind it.
- **Adaptive Logic Insights:** Specifically logs "Skipped TCP Attempts" when the system operates in UDP mode due to high congestion (e.g., "[INFO] Decision: UDP selected (80% weight). TCP handshake skipped.>").
- **Performance Metrics:** Logs the latency of the re-connection process, specifically the **SYN-ACK Time Interval** (the time elapsed between sending a SYN packet to a new dynamic port and receiving the acknowledgment).
- **Port History:** Maintains a sequential record of all computed and utilized ports for post-experiment auditing.

4.7 Module 7 — Message/Packet Format Specification

Since payload encryption is out of scope, the protocol uses a lightweight custom header structure to manage synchronization.

Field	Size (Bytes)	Type	Description
version	1	uint8	Protocol Version (e.g. 1)
msg_type	1	uint8	AUTH=1, DATA=2, ACK=3, ERROR=4
seq_num	4	uint32	Sequence number for UDP ordering
payload_len	4	uint32	Length of the message following this header

5 Low Level Design

5.1 Communication Controller (Flow)

- **1. Initialization:**
 - Load Config -> Initialize Loggers.
 - Call *Timing Module* to get initial Time-Step.
- **2. Interval Loop (Main Thread):**
 - **Check Time:** Is Current_Time_Step > Last_Time_Step?
 - **YES (Switch Event):**
 - Pause Outgoing Queue.
 - Call *Protocol Decision Logic* -> Get Target_Protocol (TCP/UDP).
 - Call *Role Assignment* -> Get Role (Listener/Dialer).
 - Call *Port Management* -> Get Target_Port.
 - Execute Switch -> Verify New Connection.
 - **Flush Buffer:** Send all queued messages to the new port.
 - Resume Outgoing Queue.
- **Handle I/O:**
 - If Incoming_Packet -> Pass to UI (Chat History).
 - If User_Input -> Push to Outgoing Queue.

5.2 Protocol Decision Logic (Flow)

- Input: Packet_Loss_Rate, Avg_Latency, Randomization-Secret-Key.
- 1. Determine Network State:
 - If Latency > 200ms OR Loss > 5% -> Set State = CONGESTED.
 - Else -> Set State = STABLE.
- 2. Set Probability Weight:
 - If STABLE -> Set UDP_Threshold = 50 (50% chance).
 - If CONGESTED -> Set UDP_Threshold = 20 (80% chance for UDP).
- 3. Generate Decision:
 - Compute Random_Value = Hash(Secret-Key+ Time_Step) % 100.
 - If Random_Value > UDP_Threshold -> Return UDP.
 - Else -> Return TCP.

5.3 Port Management Component (Flow)

- Input: Secret-Key, Time-Step, Port_Range (Min, Max).
- 1. Computation:
 - Hash_Val = SHA256(Secret-Key+ Time-Step).
 - Offset = ConvertToInt(Hash_Val) % (Max - Min).
 - Target_Port = Min + Offset.
- 2. Execution (Handover):
- Teardown: shutdown(old_socket, RDWR) -> close(old_socket).
- Re-Initialization:
 - Create new Raw Socket.
 - If Role is Listener -> bind(Target_Port).
 - If Role is Dialer -> connect(Target_IP, Target_Port).

5.4 Timing & Synchronization Handler (Flow)

- 1. Sync Check:
 - Query System_Clock (NTP Synchronized).
- 2. Step Calculation:
 - $\text{Current_Step} = \text{Floor}(\text{System_Time} / \text{Configured_Interval})$.
- 3. Trigger Logic:
 - Compare Current_Step with Stored_Step.
 - If Different -> Return TRUE (Trigger Port Switch).
 - Else -> Return FALSE.

5.5 Logging Component (Flow)

- Input: Log_Level, Event_String, Context_Data.
- 1. Filter Sensitive Data:
 - If Event_Type == AUTH_TOKEN -> ABORT (Do not log).
- 2. Console Output (Real-Time):
 - If Log_Level == INFO OR CRITICAL -> Print to Terminal Panel.
- 3. File Output (Audit Trail):
 - Open session.log in Append Mode.
 - Write: [Timestamp] | [Level] | [Active_Port] | [Protocol] | Message.
 - *Special Case*: If UDP mode active and packet loss detected -> Log "TCP Attempt Skipped due to congestion logic."

5.6 Role Assignment Component (Flow)

- **Input:** Peer_ID (0 or 1), Secret Key, Time-Step.
- **1. Deterministic Role Calculation:**
 - $\text{Global_Hash} = \text{Hash}(\text{Secret Key} + \text{Time-Step})$.
 - $\text{Turn_Indicator} = \text{Global_Hash} \% 2$. (Returns 0 or 1).
- **2. Assignment:**
 - If Turn_Indicator == Peer_ID:
 - Return **LISTENER** (Bind & Wait).
 - Else:
 - Return **DIALER** (Initiate Connection).

5.7 Packet Reliability Mechanisms (UDP Mode)

To mitigate packet loss inherent in UDP and during port transition phases, the system implements the following strategies:

- **Packet Redundancy (Dual-Send):** For critical chat messages in UDP mode, the system optionally dispatches the same datagram twice in rapid succession. This statistical approach reduces the probability of loss without requiring a complex handshake.
- **Application-Layer Acknowledgement:** Upon receiving a DATA packet, the receiver immediately dispatches an ACK packet (Message Type 3) back to the sender. If the sender does not receive an ACK within a defined timeout (e.g., 500ms), the message is re-transmitted.

- **Transition Buffering:** During the "Handover Phase" (port switching), the *Communication Controller* pauses the outgoing transmission queue. Messages generated during this micro-window are buffered in local memory and flushed to the new port once the re-handshake is verified.

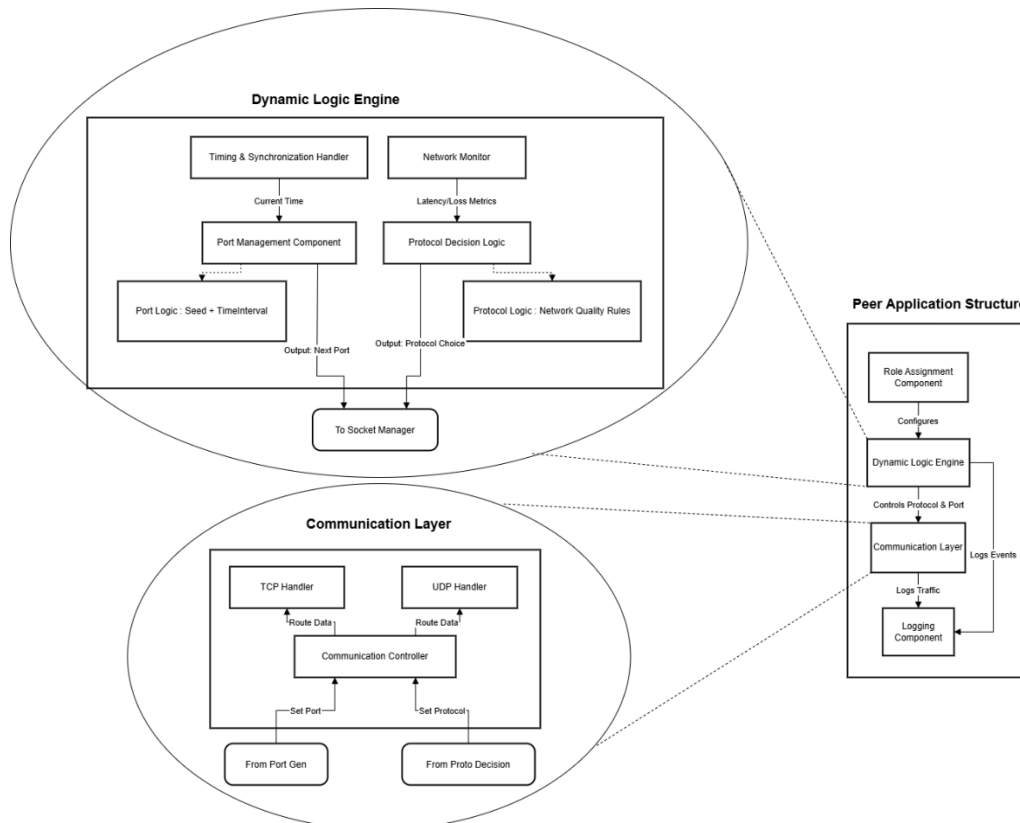


Diagram 2: Application Architecture

6 References

The resources used to produce this document should be listed here and referenced in the relevant text of this document

- [1] Çankaya University Software Engineering Department, "Graduation Projects," [Online]. Available: <https://seng.cankaya.edu.tr/graduation-projects/>. [Accessed: 28 Dec 2025].
- [2] W. R. Stevens, B. Fenner, and A. M. Rudoff, UNIX Network Programming, Volume 1: The Sockets Networking API, 3rd ed. Addison-Wesley Professional, 2003.
- [3] National Institute of Standards and Technology (NIST), "FIPS PUB 180-4: Secure Hash Standard (SHS)," 2015. [Online]. Available: <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.180-4.pdf>
- [4] J. H. Jafarian, E. Al-Shaer, and Q. Duan, "Openflow-based user-level moving target defense," INMN '12, 2012.
- [5] B. Hall, "Beej's Guide to Network Programming," [Online]. Available: <https://beej.us/guide/bgnet/>
- [6] <https://link.springer.com/article/10.1631/FITEE.1601321#:~:text=,383>

- [7]<https://www.mdpi.com/2079-9292/11/18/2886#:~:text=A%20Survey%20on%20Moving%20Target,for%20Networks%3A%20A%20Practical%20View>
- [8]<https://www.mdpi.com/2079-9292/11/18/2886#:~:text=93,CrossRef>
- [9]https://www.usenix.org/legacy/event/usenix05/tech/general/full_papers/ford/ford.pdf#:~:text=USENIX%20Association%202005%20USENIX%20Annual,com%20Dan%20Kegel
- [10]https://beej.us/guide/bgnet/pdf/bgnet_a4_c_1.pdf#:~:text=%5BPDF%5D%20bgnet_a4_c_1.pdf%20,and%20translations%2C%20below%2C%20this
- [11]<https://www.geeksforgeeks.org/c/socket-programming-cc/>
- [12]<https://www.geeksforgeeks.org/computer-networks/p2p-peer-to-peer-file-sharing/>
- [13]<https://www.geeksforgeeks.org/c/tcp-server-client-implementation-in-c/>
- [14]<https://beej.us/guide/bgnet/html/split/#:~:text=Beej%27s%20Guide%20to%20Network%20Programming>
- [15]<https://www.geeksforgeeks.org/computer-networks/nat-hole-punching-in-computer-network/>